WSQ COURSE SLIDES

AI Vibe Coding for SQL

TGS-2021002619

Prompt · Query · Validate · Ship

Version v1.0 · 27 August 2026



ONE DAY · 8 TRAINING HOURS

Course Navigation

1

1 · Model

Prompt contracts, entities, keys and constraints

2

2 · Query

CRUD, filters, joins and safe parameters

3

3 · Transform

Quality, aggregation, CTEs, transactions and indexes

4

4 · Deliver

Warehouse mappings, dashboards and capstone evidence

TRAQOM · MANDATORY

Digital Attendance

1

AM

Scan the SSG attendance QR at course start

2

PM

Complete the afternoon attendance checkpoint

3

Assessment

Scan again before the assessment

4

Eligibility

At least 75% attendance and competent assessment

About the Trainer



Your Trainer

Complete this profile before delivery

NAME _____

QUALIFICATIONS _____

SQL / AI EXPERIENCE _____

CONTACT _____

TRAINER PROFILE

About the Trainer



Dr Alfred Ang

Principal Trainer · Tertiary Infotech Academy

DOMAIN	Data science, AI, optimisation and database systems
TEACHING	WSQ technical courses and applied project learning
APPROACH	Mechanism first: prompt, inspect, execute, test, explain
EVIDENCE	Queries, plans, reconciliations and working applications

ICEBREAKER

Know Your Starting Point

1

Role

What decisions do you make with data?

2

SQL

Which queries can you already read?

3

AI

Which assistant do you use and how do you validate it?

4

Outcome

Which small database app would save you time?

WORKING AGREEMENT

Ground Rules

1

Protect data

Use only fictional lab records

2

Challenge AI

Never accept generated SQL without tests

3

Pair and explain

Share reasoning, not just copied output

4

Save evidence

Keep prompts, query results and checks

LMS / TMS VISUAL PATH

Download Course Materials

1

Sign in

Open lms-tms.tertiaryinfotech.com

2

Open course

Select AI Vibe Coding for SQL

3

Download

Trainer PDF, Learner Guide and Labs

4

Verify

Confirm 11 lab folders and assessment links

Learning Outcomes

1

LO1

Apply data modelling

2

LO2

Apply data processing and analysis

3

LO3

Apply data aggregation and transformation

4

LO4

Perform data mapping to a data warehouse

8 TRAINING HOURS + 1 HOUR ASSESSMENT

One-Day Learning Journey

1

09:00

Admin + AI-SQL control loop

2

10:00

Topic 1 · Model

3

12:00

Topic 2 · Query

4

14:45

Topics 3–4 · Transform + deliver

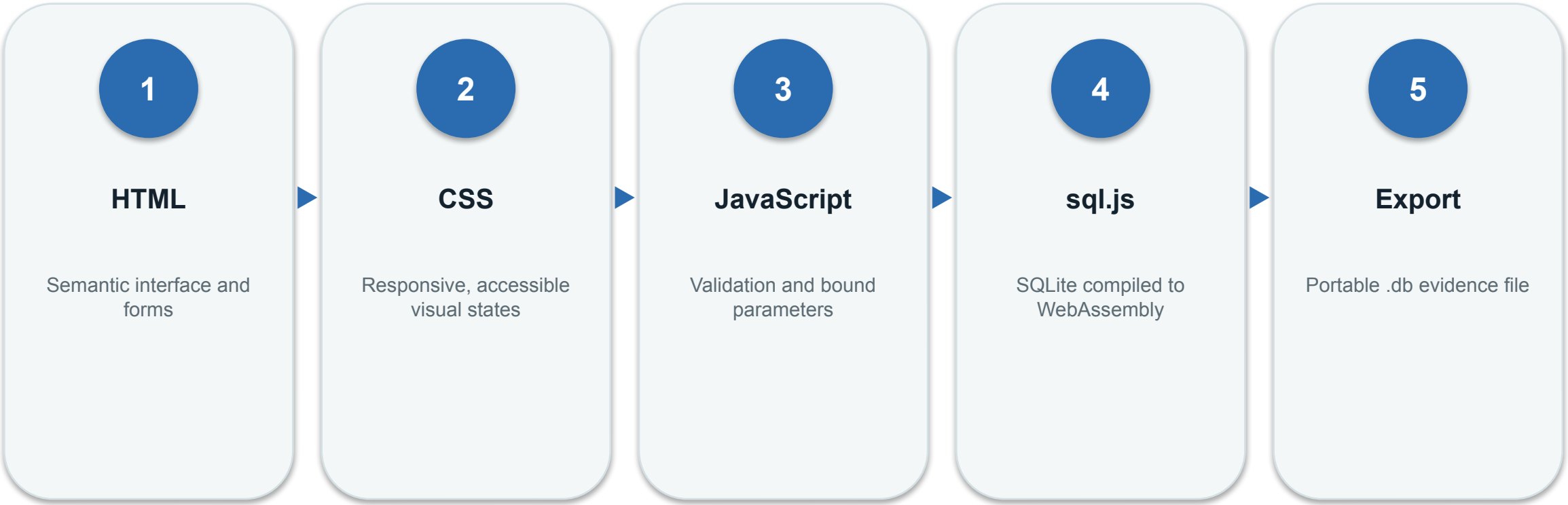
5

17:30

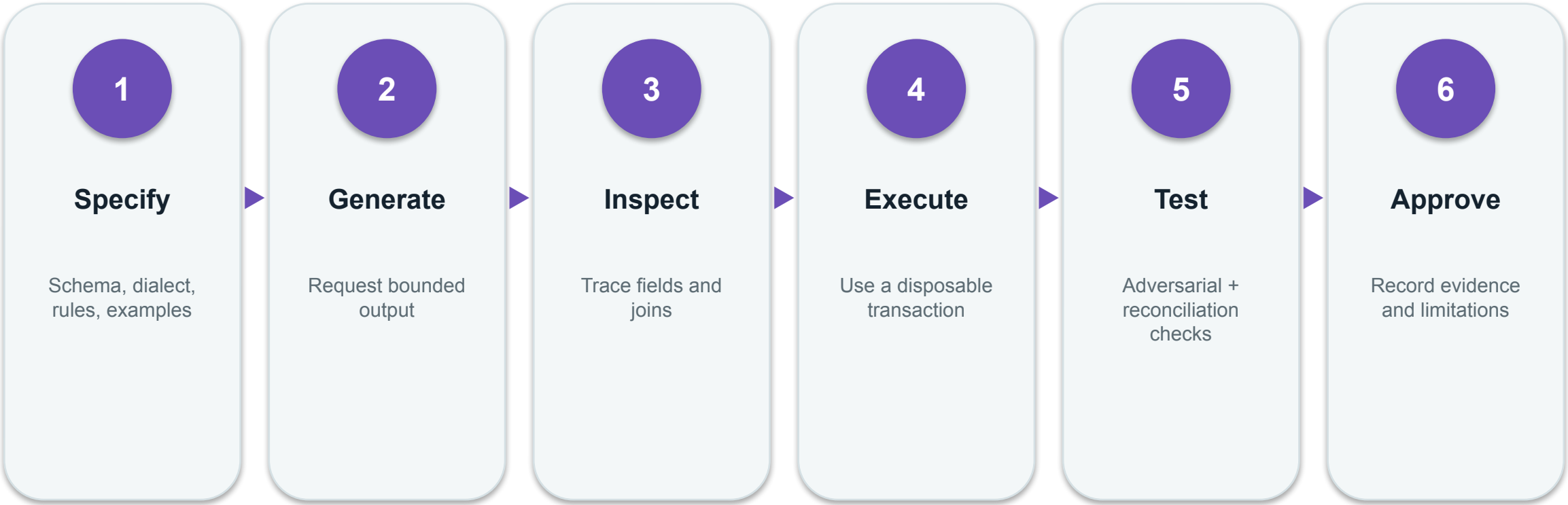
WA + PP assessment

LAB ARCHITECTURE

Browser SQLite Runtime



AI-to-SQL Control Loop



BEFORE ASSESSMENT

Assessment Briefing

1

Open book

Use approved slides, guide and lab assets

2

Individual

No discussion or recording

3

Submission

Upload completed answers to the LMS

4

Competency

Every mapped K and A criterion must pass

WA + PP · 60 MINUTES

Assessment

1

Written · 20 min

7 open-ended questions covering K1–K7

2

Practical · 40 min

4 connected tasks covering A1–A9

3

Evidence

SQL, output, validation and explanation

4

Result

Competent / Not Yet Competent

ON ASSESSMENT DAY

Assessment Flow

1

TRAQOM

Assessment attendance

2

WA

7 knowledge responses

3

PP

4 practical SQL tasks

4

Upload

Submit on LMS

5

Sign-off

Review outcome and appeal route

Topic 01 · AI-Assisted SQL Data Modelling and Database Design

01

Requirements to normalised relational schemas, constraints and safe AI prompt contracts

Requirement contract

THE CONTRACT

Entities, attributes, relationships, invariants and sample rows become explicit prompt inputs.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

Relational integrity

THE CONTRACT

PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE and CHECK constraints reject invalid states.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

Normalisation

THE CONTRACT

1NF removes repeating groups; 2NF removes partial dependency; 3NF removes transitive dependency.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1

MECHANISM Trace the row or state change inside SQLite.

2

CONTROL Name the constraint, predicate or transaction boundary.

3

EVIDENCE Retain SQL, output, a failure test and reconciliation.

AI validation gate

THE CONTRACT

Treat generated DDL as a proposal: inspect, execute in a transaction, test constraints, then approve.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

MECHANISM

Relational Integrity Boundary



REALISTIC BUILD SCENARIO

Lab 1 · Prompt-to-Schema Design Studio

Business request

Model a Singapore retail ordering workflow from an ambiguous business brief.

LO1 · K1–K3 · A1–A4

PROMPT

Make requirements explicit

BUILD

Generate and correct SQL

TEST

Attack assumptions

EVIDENCE

Record accepted output

Lab 1 · Data Path



Lab 1 · Prompt Contract

Act as a senior SQLite data modeller. From the requirements below, return: assumptions; entities and cardinalities; 3NF tables; SQLite DDL with explicit PK, FK, NOT NULL, UNIQUE and CHECK constraints; five adversarial test cases. Do not invent fields. Mark every uncertainty as NEEDS_CONFIRMATION.

INPUT	GENERATE	VERIFY
Known schema	SQLite only	Tests + evidence

Context

Schema + business rule

Contract

Exact SQL dialect and output

Guardrail

No invented fields

Evidence

Tests before approval

Lab 1 · Worked Query

```
SELECT name, sql FROM sqlite_master WHERE
type='table' ORDER BY name;
```

INPUT ROWSET ▶ SQL ENGINE ▶ ACCEPTED RESULT

Predict the row shape before execution; reconcile it after execution.

What to inspect

1. Check input grain
2. Inspect join and predicate
3. Predict result shape
4. Reconcile output

ACCEPTANCE EVIDENCE

Lab 1 · Verification and Failure

✓ Acceptance check

Invalid email, negative price and orphan order rows are rejected; valid sample rows insert successfully.

! Failure diagnostic

AI invents fields or omits constraints → restate the data contract and require a requirement-to-column trace.

REALISTIC BUILD SCENARIO

Lab 2 · SQLite To-Do List Web App

Business request

Build a personal work queue with priorities, due dates and completion state.

LO1 · LO2 · A1–A6

PROMPT

Make requirements explicit

BUILD

Generate and correct SQL

TEST

Attack assumptions

EVIDENCE

Record accepted output

A RESPONSIVE HTML/CSS/JAVASCRIPT TO-DO APP THAT PERSISTS AN EXPORTED SQLITE DATABASE.

Lab 2 · Data Path



Lab 2 · Prompt Contract

Generate SQLite DDL and parameterised CRUD statements for a tasks table with title, priority, due_date, completed and created_at. Enforce allowed priorities and completed as 0/1. Return SQL only, then list five tests separately.

INPUT	GENERATE	VERIFY
Known schema	SQLite only	Tests + evidence

Context

Schema + business rule

Contract

Exact SQL dialect and output

Guardrail

No invented fields

Evidence

Tests before approval

Lab 2 · Worked Query

```
SELECT id,title,priority,due_date,completed FROM
tasks ORDER BY completed, CASE priority WHEN 'High'
THEN 1 WHEN 'Medium' THEN 2 ELSE 3 END, due_date;
```

INPUT ROWSET ▶ SQL ENGINE ▶ ACCEPTED RESULT

Predict the row shape before execution; reconcile it after execution.

What to inspect

1. Check input grain
2. Inspect join and predicate
3. Predict result shape
4. Reconcile output

ACCEPTANCE EVIDENCE

Lab 2 · Verification and Failure

✓ Acceptance check

Add, complete and delete tasks; export the database, reload the page, import it and recover the same rows.

! Failure diagnostic

Unsafe string concatenation or missing domain checks → replace with bound parameters and CHECK constraints.

REALISTIC BUILD SCENARIO

Lab 3 · Customer and Order Relationship Lab

Business request

Turn spreadsheet-style customer and order data into a clean relational model.

LO1 · K2–K3 · A1–A4

PROMPT

Make requirements explicit

BUILD

Generate and correct SQL

TEST

Attack assumptions

EVIDENCE

Record accepted output

CUSTOMER, ORDER AND ORDER-ITEM TABLES WITH TESTED ONE-TO-MANY RELATIONSHIPS.

Lab 3 · Data Path



Lab 3 · Prompt Contract

Review these customer/order columns. Identify repeating groups and dependencies, propose a 3NF SQLite design, state the grain of each table, and provide DDL plus orphan-key tests. Preserve source identifiers for reconciliation.

INPUT	GENERATE	VERIFY
Known schema	SQLite only	Tests + evidence

Context

Schema + business rule

Contract

Exact SQL dialect and output

Guardrail

No invented fields

Evidence

Tests before approval

Lab 3 · Worked Query

```
SELECT c.name, COUNT(DISTINCT o.id) orders,
ROUND(SUM(oi.qty*oi.unit_price),2) revenue FROM
customers c JOIN orders o ON o.customer_id=c.id JOIN
order_items oi ON oi.order_id=o.id GROUP BY
c.id,c.name ORDER BY revenue DESC;
```

INPUT ROWSET ▶ SQL ENGINE ▶ ACCEPTED RESULT

Predict the row shape before execution; reconcile it after execution.

What to inspect

1. Check input grain
2. Inspect join and predicate
3. Predict result shape
4. Reconcile output

ACCEPTANCE EVIDENCE

Lab 3 · Verification and Failure

✓ Acceptance check

The revenue total matches the seed control total and every order item resolves to one order and one product.

! Failure diagnostic

Join duplicates inflate revenue → aggregate at order-item grain and verify keys before summing.

NATIVE POWERPOINT CHART

Topic 01 · Evidence Chart



What the data shows

The chart is editable and bound to the same metric grain as the SQL. Validate totals before interpreting differences; visual polish never substitutes for reconciliation.

RECAP

Topic 01 · Decision Checklist

1

Contract

Can you state grain, keys and rules?

2

Mechanism

Can you trace runtime row flow?

3

Control

Can you explain the failure path?

4

Evidence

Can you reproduce and reconcile output?

Topic 02 · SQL Data Processing, Querying and Analysis

02

CRUD, predicates, joins, parameter binding and evidence-led correction of AI-generated queries

Query pipeline

THE CONTRACT

FROM/JOIN builds the rowset; WHERE filters; GROUP BY aggregates; HAVING filters groups; SELECT projects; ORDER BY sorts.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

Safe parameters

THE CONTRACT

Bind user input as values instead of concatenating strings; structure such as column names stays allow-listed.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

Join cardinality

THE CONTRACT

Keys and relationship direction predict row counts; unexpected multiplication is a diagnostic signal.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

CRUD control

THE CONTRACT

Validate inputs, wrap dependent writes in a transaction, verify affected rows and preserve an audit trail.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

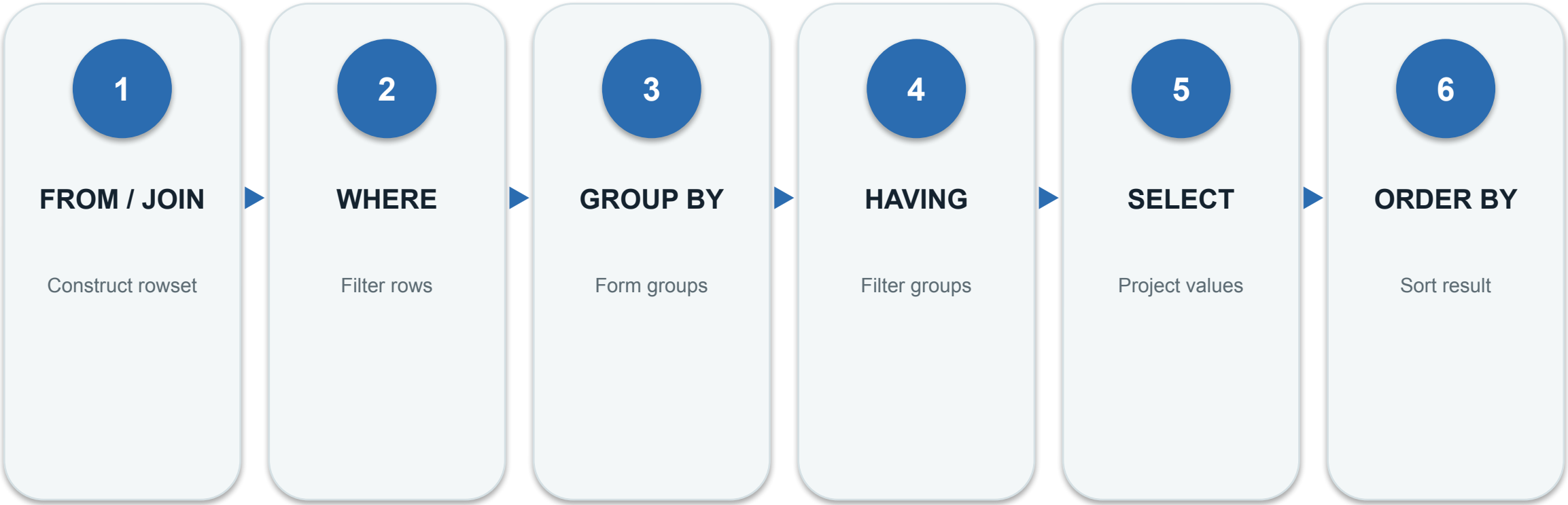
1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

EXECUTION ORDER

Logical Query Processing



REALISTIC BUILD SCENARIO

Lab 4 · Inventory CRUD Web App

Business request

Manage stock levels for a small Singapore retailer without allowing negative inventory.

LO2 · K4 · A5–A6

PROMPT

Make requirements explicit

BUILD

Generate and correct SQL

TEST

Attack assumptions

EVIDENCE

Record accepted output

A SEARCHABLE INVENTORY INTERFACE WITH PARAMETERISED STOCK ADJUSTMENTS AND LOW-STOCK FLAGS.

Lab 4 · Data Path



Lab 4 · Prompt Contract

Produce parameterised SQLite CRUD statements for products. A stock adjustment must be atomic, reject negative resulting stock, and return the updated row. Include a transaction and rollback path.

INPUT	GENERATE	VERIFY
Known schema	SQLite only	Tests + evidence

Context

Schema + business rule

Contract

Exact SQL dialect and output

Guardrail

No invented fields

Evidence

Tests before approval

Lab 4 · Worked Query

```
SELECT sku,name,stock,reorder_level, CASE WHEN
stock<=reorder_level THEN 'REORDER' ELSE 'OK' END
status FROM products ORDER BY status DESC, stock;
```

INPUT ROWSET ▶ SQL ENGINE ▶ ACCEPTED RESULT

Predict the row shape before execution; reconcile it after execution.

What to inspect

1. Check input grain
2. Inspect join and predicate
3. Predict result shape
4. Reconcile output

ACCEPTANCE EVIDENCE

Lab 4 · Verification and Failure

✓ Acceptance check

Positive and negative adjustments update one row; an adjustment below zero is rolled back.

! Failure diagnostic

Read-modify-write race or negative balance → use one guarded UPDATE inside a transaction and check changes().

REALISTIC BUILD SCENARIO

Lab 5 · Order Query and Filter Explorer

Business request

Answer operations questions using SELECT, WHERE, ORDER BY, LIMIT and date filters.

LO2 · K4 · A5–A6

PROMPT

Make requirements explicit

BUILD

Generate and correct SQL

TEST

Attack assumptions

EVIDENCE

Record accepted output

A TESTED LIBRARY OF PARAMETERISED OPERATIONAL QUERIES AND EXPECTED RESULT SETS.

Lab 5 · Data Path



Lab 5 · Prompt Contract

Given this schema and business question, generate one SQLite SELECT using named parameters. Explain predicate selectivity and null behaviour. Provide a second query that proves the result is complete.

INPUT	GENERATE	VERIFY
Known schema	SQLite only	Tests + evidence

Context

Schema + business rule

Contract

Exact SQL dialect and output

Guardrail

No invented fields

Evidence

Tests before approval

Lab 5 · Worked Query

```
SELECT
order_no,customer_name,status,total,order_date FROM
orders WHERE status='Paid' AND order_date>='2026-08-
01' ORDER BY total DESC LIMIT 10;
```

INPUT ROWSET ▶ SQL ENGINE ▶ ACCEPTED RESULT

Predict the row shape before execution; reconcile it after execution.

What to inspect

1. Check input grain
2. Inspect join and predicate
3. Predict result shape
4. Reconcile output

ACCEPTANCE EVIDENCE

Lab 5 · Verification and Failure

✓ Acceptance check

Results satisfy every predicate, sort descending and match an independent COUNT control query.

! Failure diagnostic

NULL rows disappear unexpectedly → state three-valued logic and test IS NULL explicitly.

REALISTIC BUILD SCENARIO

Lab 6 · Service Desk Join Lab

Business request

Combine tickets, requesters and agents without losing unassigned tickets.

LO2 · K3–K4 · A3–A6

PROMPT

Make requirements explicit

BUILD

Generate and correct SQL

TEST

Attack assumptions

EVIDENCE

Record accepted output

A SERVICE-DESK BROWSER APP AND JOIN REPORT THAT PRESERVES UNMATCHED ROWS.

Lab 6 · Data Path



Lab 6 · Prompt Contract

Write a SQLite query for all tickets with requester and optional assigned agent. Preserve unassigned tickets, derive age_hours, and order SLA breaches first. Explain why each join is INNER or LEFT.

INPUT	GENERATE	VERIFY
Known schema	SQLite only	Tests + evidence

Context

Schema + business rule

Contract

Exact SQL dialect and output

Guardrail

No invented fields

Evidence

Tests before approval

Lab 6 · Worked Query

```
SELECT t.id,t.subject,r.name requester,a.name
agent,ROUND((julianday('now')-
julianday(t.created_at))*24,1) age_hours FROM
tickets t JOIN people r ON r.id=t.requester_id LEFT
JOIN people a ON a.id=t.agent_id ORDER BY
(age_hours>24) DESC,age_hours DESC;
```

INPUT ROWSET ▶ SQL ENGINE ▶ ACCEPTED RESULT

Predict the row shape before execution; reconcile it after execution.

What to inspect

1. Check input grain
2. Inspect join and predicate
3. Predict result shape
4. Reconcile output

ACCEPTANCE EVIDENCE

Lab 6 · Verification and Failure

✓ Acceptance check

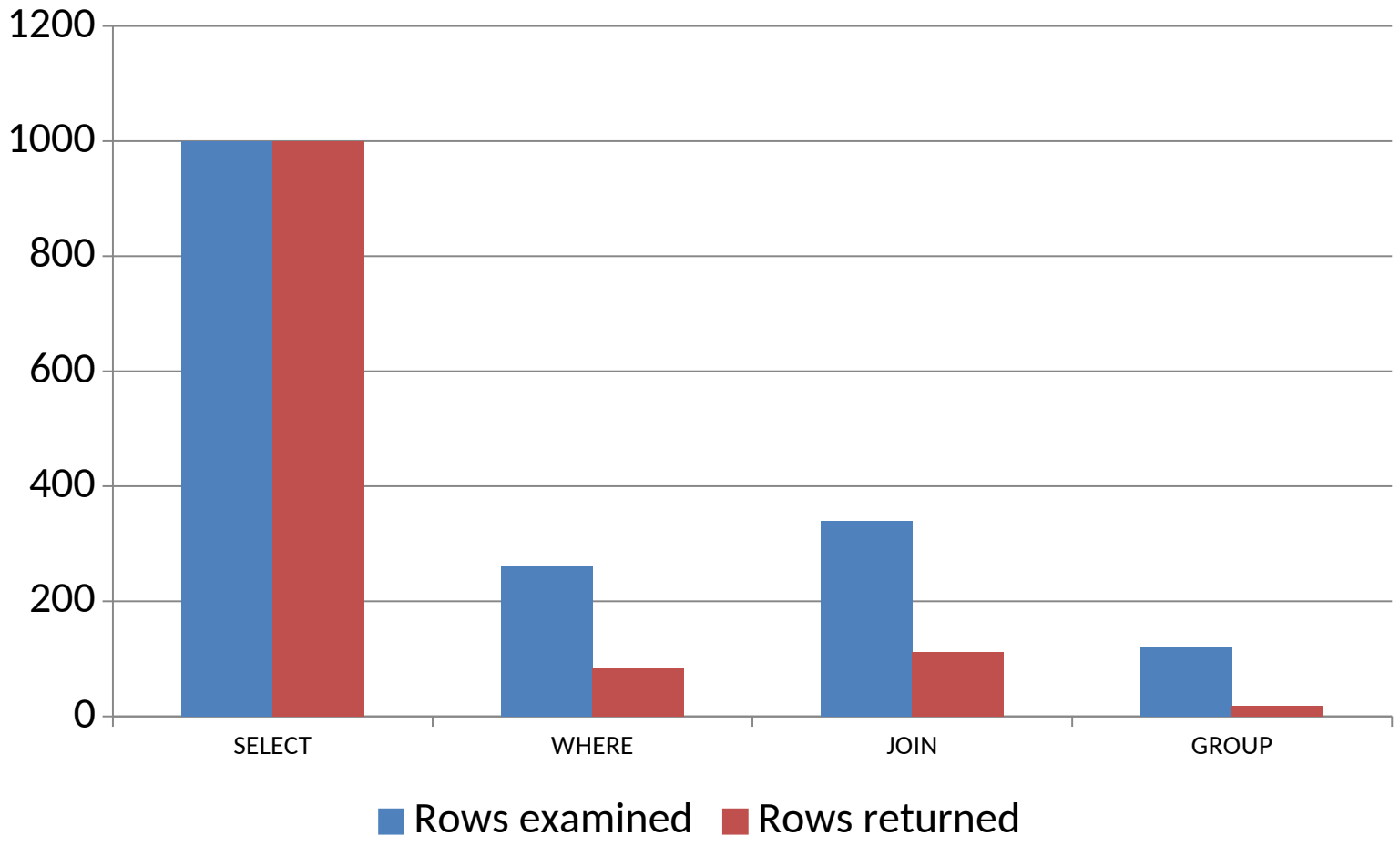
Every ticket appears exactly once and unassigned tickets show a blank agent rather than disappearing.

! Failure diagnostic

INNER JOIN removes optional relationships → change only the optional side to LEFT JOIN and test row counts.

NATIVE POWERPOINT CHART

Topic 02 · Evidence Chart



What the data shows

The chart is editable and bound to the same metric grain as the SQL. Validate totals before interpreting differences; visual polish never substitutes for reconciliation.

RECAP

Topic 02 · Decision Checklist

1

Contract

Can you state grain, keys and rules?

2

Mechanism

Can you trace runtime row flow?

3

Control

Can you explain the failure path?

4

Evidence

Can you reproduce and reconcile output?

Topic 03 · SQL Data Aggregation and Transformation

03

Aggregate measures, CTEs, data-quality rules, transactions, indexes and query plans

Grain first

THE CONTRACT

Define what one output row represents before choosing GROUP BY columns or joining detail tables.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

Quality profile

THE CONTRACT

Count nulls, duplicates, invalid domains and orphan keys before transforming source data.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

Atomic change

THE CONTRACT

BEGIN, COMMIT and ROLLBACK protect multi-step updates from partial completion.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

Performance evidence

THE CONTRACT

EXPLAIN QUERY PLAN proves whether an index changes access from SCAN to SEARCH.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

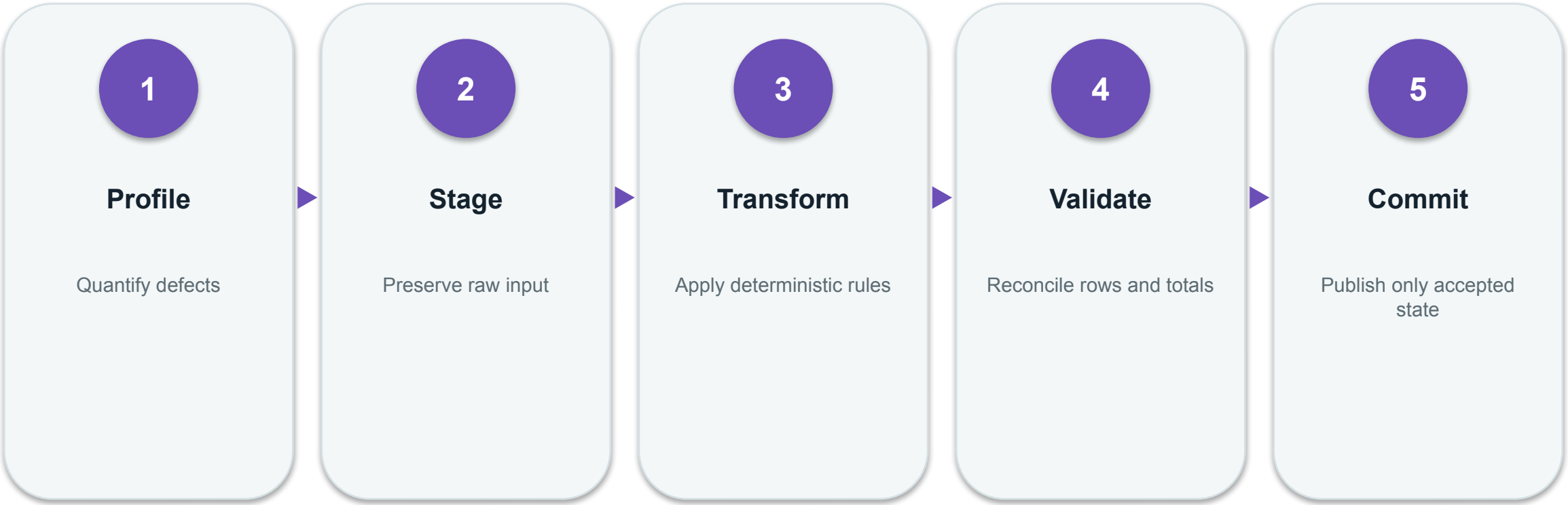
1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

DATA PIPELINE

Transformation Control



REALISTIC BUILD SCENARIO

Lab 7 · Sales Aggregation Dashboard

Business request

Create management KPIs from order-line data at a declared grain.

LO3 · K5–K6 · A5–A7

PROMPT

Make requirements explicit

BUILD

Generate and correct SQL

TEST

Attack assumptions

EVIDENCE

Record accepted output

A BROWSER DASHBOARD WITH REVENUE, AVERAGE ORDER VALUE AND CATEGORY PERFORMANCE.

Lab 7 · Data Path



Lab 7 · Prompt Contract

At order-line grain, generate SQLite queries for revenue, distinct orders, average order value and monthly category totals. Prevent double counting and define every numerator and denominator.

INPUT	GENERATE	VERIFY
Known schema	SQLite only	Tests + evidence

Context

Schema + business rule

Contract

Exact SQL dialect and output

Guardrail

No invented fields

Evidence

Tests before approval

Lab 7 · Worked Query

```
WITH line AS (SELECT
oi.order_id,p.category,oi.qty*oi.unit_price amount
FROM order_items oi JOIN products p ON
p.id=oi.product_id) SELECT
category,ROUND(SUM(amount),2) revenue,COUNT(DISTINCT
order_id) orders FROM line GROUP BY category ORDER
BY revenue DESC;
```

INPUT ROWSET ▶ SQL ENGINE ▶ ACCEPTED RESULT

Predict the row shape before execution; reconcile it after execution.

What to inspect

1. Check input grain
2. Inspect join and predicate
3. Predict result shape
4. Reconcile output

ACCEPTANCE EVIDENCE

Lab 7 · Verification and Failure

✓ Acceptance check

Dashboard grand total equals SUM(qty*unit_price) from order_items and AOV equals revenue/distinct orders.

! Failure diagnostic

Averages use line count instead of order count → declare grain and use COUNT(DISTINCT order_id).

REALISTIC BUILD SCENARIO

Lab 8 · Data Quality and Transaction Control

Business request

Clean supplier data and update linked records without partial writes.

LO3 · K5–K6 · A3–A7

PROMPT

Make requirements explicit

BUILD

Generate and correct SQL

TEST

Attack assumptions

EVIDENCE

Record accepted output

A PROFILING REPORT, REJECT TABLE AND ATOMIC CORRECTION TRANSACTION.

Lab 8 · Data Path

1

Browser UI

HTML/CSS captures intent

2

Controller

JavaScript validates and binds

3

SQLite WASM

DDL, query and transaction

4

Evidence

Rows, plan and acceptance check

Lab 8 · Prompt Contract

Profile this SQLite supplier table for nulls, duplicate emails, invalid status values and orphan product references. Propose deterministic corrections, a reject table and one atomic transaction. Never delete source rows silently.

INPUT	GENERATE	VERIFY
Known schema	SQLite only	Tests + evidence

Context

Schema + business rule

Contract

Exact SQL dialect and output

Guardrail

No invented fields

Evidence

Tests before approval

Lab 8 · Worked Query

```
SELECT 'null_email' issue,COUNT(*) n FROM suppliers
WHERE email IS NULL UNION ALL SELECT
'duplicate_email',COUNT(*) FROM (SELECT email FROM
suppliers WHERE email IS NOT NULL GROUP BY email
HAVING COUNT(*)>1);
```

INPUT ROWSET ▶ SQL ENGINE ▶ ACCEPTED RESULT

Predict the row shape before execution; reconcile it after execution.

What to inspect

1. Check input grain
2. Inspect join and predicate
3. Predict result shape
4. Reconcile output

ACCEPTANCE EVIDENCE

Lab 8 · Verification and Failure

✓ Acceptance check

The transaction either applies all linked corrections or rolls back; source and reject totals reconcile.

! Failure diagnostic

AI overwrites evidence or applies non-deterministic fixes → retain raw values, reason codes and explicit rules.

REALISTIC BUILD SCENARIO

Lab 9 · CTEs, Indexes and Query Plans

Business request

Improve a slow order-history query using evidence rather than intuition.

LO3 · K7 · A5–A7

PROMPT

Make requirements explicit

BUILD

Generate and correct SQL

TEST

Attack assumptions

EVIDENCE

Record accepted output

A READABLE CTE QUERY, COMPOSITE INDEX AND BEFORE/AFTER EXPLAIN QUERY PLAN EVIDENCE.

Lab 9 · Data Path



Lab 9 · Prompt Contract

Refactor this SQLite query into named CTEs, then propose the smallest useful index. Explain column order from equality, range and ordering predicates. Require EXPLAIN QUERY PLAN evidence before claiming improvement.

INPUT	GENERATE	VERIFY
Known schema	SQLite only	Tests + evidence

Context

Schema + business rule

Contract

Exact SQL dialect and output

Guardrail

No invented fields

Evidence

Tests before approval

Lab 9 · Worked Query

```
EXPLAIN QUERY PLAN SELECT
customer_id,order_date,total FROM orders WHERE
customer_id=7 AND order_date>='2026-01-01' ORDER BY
order_date DESC;
```

INPUT ROWSET ▶ SQL ENGINE ▶ ACCEPTED RESULT

Predict the row shape before execution; reconcile it after execution.

What to inspect

1. Check input grain
2. Inspect join and predicate
3. Predict result shape
4. Reconcile output

ACCEPTANCE EVIDENCE

Lab 9 · Verification and Failure

✓ Acceptance check

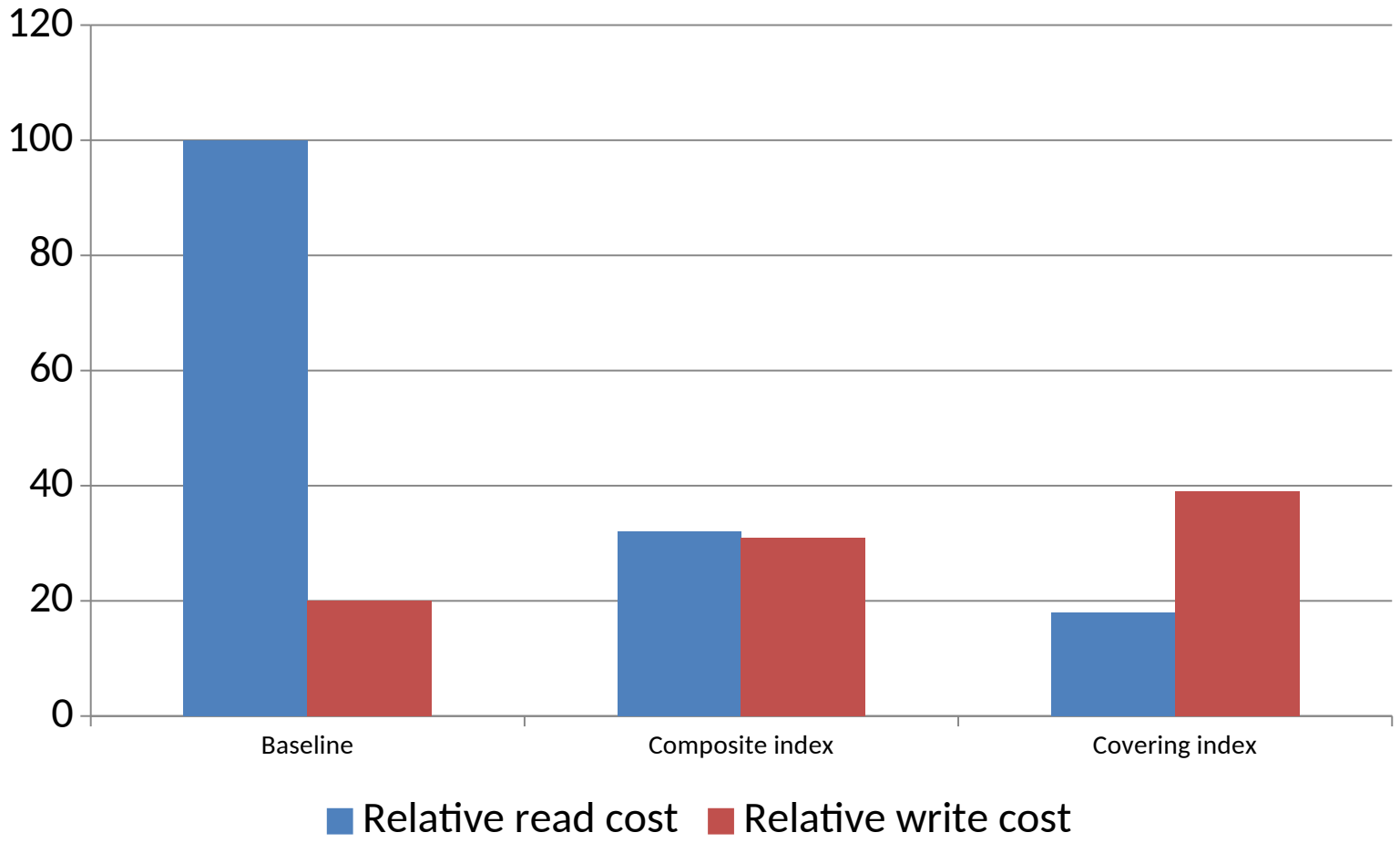
Plan changes from full SCAN to indexed SEARCH and the result set remains byte-for-byte equivalent.

! Failure diagnostic

Index adds storage but is unused → compare plan, predicate order and selectivity; remove redundant index.

NATIVE POWERPOINT CHART

Topic 03 · Evidence Chart



What the data shows

The chart is editable and bound to the same metric grain as the SQL. Validate totals before interpreting differences; visual polish never substitutes for reconciliation.

RECAP

Topic 03 · Decision Checklist

1

Contract

Can you state grain, keys and rules?

2

Mechanism

Can you trace runtime row flow?

3

Control

Can you explain the failure path?

4

Evidence

Can you reproduce and reconcile output?

Topic 04 · Data Mapping, Warehousing and Dashboard Development

04

Source-to-target mapping, star schemas, dimensional loads and browser dashboards backed by SQLite

Warehouse grain

THE CONTRACT

A fact table states one measurable event; dimensions provide stable descriptive context.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

Source-to-target map

THE CONTRACT

Each target column records source, transform, data type, null rule and reconciliation test.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

Load discipline

THE CONTRACT

Load dimensions before facts, resolve surrogate keys, quarantine rejects and reconcile totals.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

Dashboard contract

THE CONTRACT

Each visual owns a metric definition, SQL query, filter behaviour and acceptance threshold.

State it precisely enough that an AI assistant cannot invent missing columns, keys or business rules.

1 **MECHANISM** Trace the row or state change inside SQLite.

2 **CONTROL** Name the constraint, predicate or transaction boundary.

3 **EVIDENCE** Retain SQL, output, a failure test and reconciliation.

Star Schema Load



REALISTIC BUILD SCENARIO

Lab 10 · Warehouse Star Schema Dashboard

Business request

Map retail transactions into a small analytical star schema.

LO4 · K3,K7 · A7–A9

PROMPT

Make requirements explicit

BUILD

Generate and correct SQL

TEST

Attack assumptions

EVIDENCE

Record accepted output

Lab 10 · Data Path



Lab 10 · Prompt Contract

Design a SQLite star schema for retail sales at order-line grain. Return source-to-target mappings, surrogate-key strategy, dimension-first load SQL, reject handling and three reconciliation queries. Distinguish additive from non-additive measures.

INPUT	GENERATE	VERIFY
Known schema	SQLite only	Tests + evidence

Context

Schema + business rule

Contract

Exact SQL dialect and output

Guardrail

No invented fields

Evidence

Tests before approval

Lab 10 · Worked Query

```
SELECT
d.month_name,p.category,ROUND(SUM(f.sales_amount),2)
sales FROM fact_sales f JOIN dim_date d ON
d.date_key=f.date_key JOIN dim_product p ON
p.product_key=f.product_key GROUP BY
d.month_name,p.category ORDER BY
d.date_key,p.category;
```

INPUT ROWSET ▶ SQL ENGINE ▶ ACCEPTED RESULT

Predict the row shape before execution; reconcile it after execution.

What to inspect

1. Check input grain
2. Inspect join and predicate
3. Predict result shape
4. Reconcile output

ACCEPTANCE EVIDENCE

Lab 10 · Verification and Failure

✓ Acceptance check

Fact row count equals accepted source lines; fact amount reconciles to source amount; no unknown foreign keys remain.

! Failure diagnostic

Facts load before dimensions or grain is unclear → resolve dimension keys first and reject unresolved rows.

REALISTIC BUILD SCENARIO

Lab 11 · Capstone: AI-Built Service Operations App

Business request

Deliver a small service-operations product from brief to validated dashboard.

LO1–LO4 · K1–K7 · A1–A9

PROMPT

Make requirements explicit

BUILD

Generate and correct SQL

TEST

Attack assumptions

EVIDENCE

Record accepted output

Lab 11 · Data Path

1

Browser UI

HTML/CSS captures intent

2

Controller

JavaScript validates and binds

3

SQLite WASM

DDL, query and transaction

4

Evidence

Rows, plan and acceptance check

Lab 11 · Prompt Contract

You are my SQL pair programmer. Work in stages and wait for approval after each: clarify requirements; propose 3NF model; generate SQLite DDL; generate parameterised CRUD; generate SLA report; generate tests and threat review. For every stage, list assumptions, expected outputs and failure tests. Do not change approved contracts silently.

INPUT	GENERATE	VERIFY
Known schema	SQLite only	Tests + evidence

Context

Schema + business rule

Contract

Exact SQL dialect and output

Guardrail

No invented fields

Evidence

Tests before approval

Lab 11 · Worked Query

```
SELECT status,priority,COUNT(*)
tickets,ROUND(AVG((julianday(COALESCE(closed_at,'now
'))-julianday(created_at))*24),1) avg_hours FROM
tickets GROUP BY status,priority ORDER BY
status,priority;
```

INPUT ROWSET ▶ SQL ENGINE ▶ ACCEPTED RESULT

Predict the row shape before execution; reconcile it after execution.

What to inspect

1. Check input grain
2. Inspect join and predicate
3. Predict result shape
4. Reconcile output

ACCEPTANCE EVIDENCE

Lab 11 · Verification and Failure

✓ Acceptance check

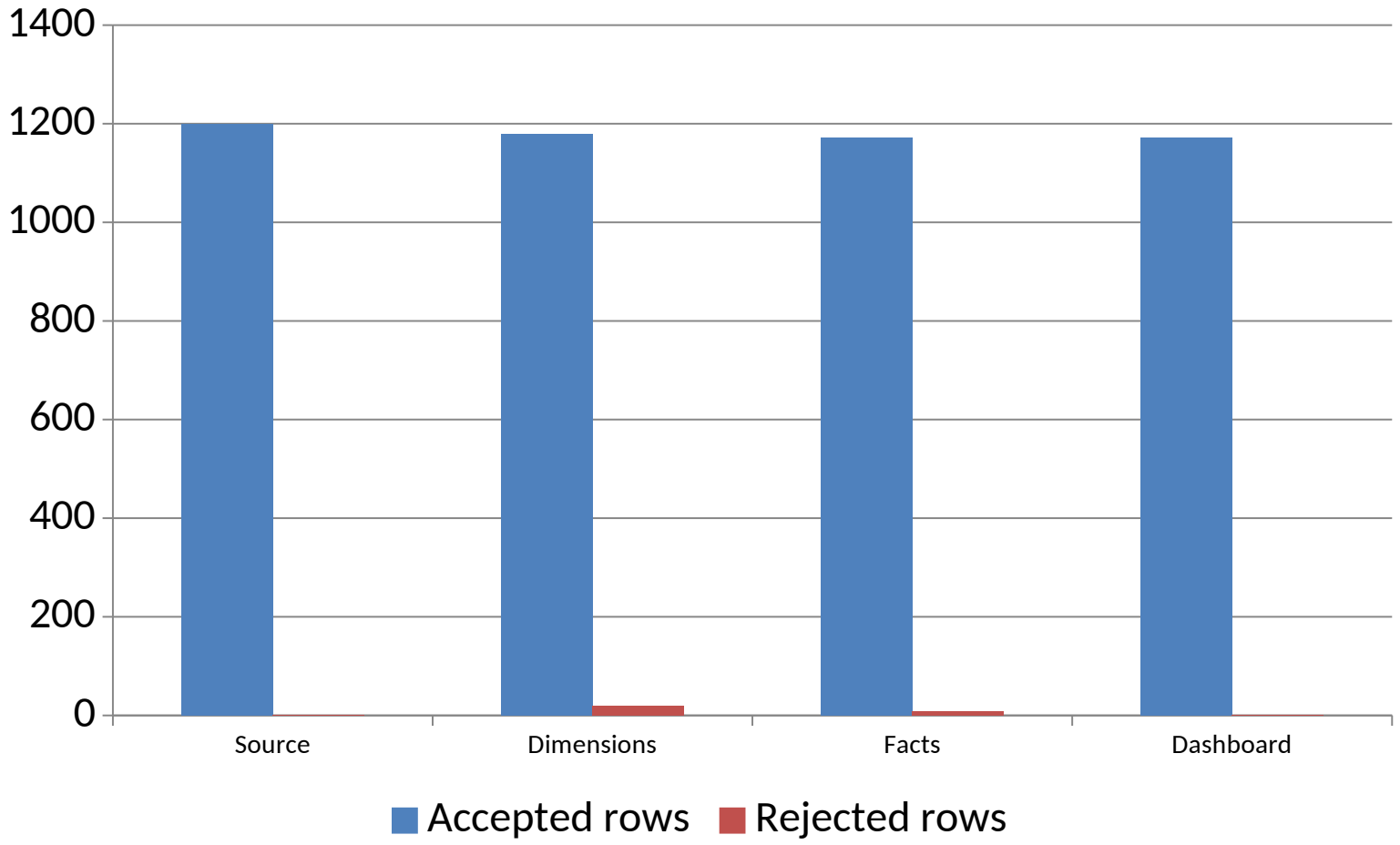
All acceptance tests pass: integrity, parameter binding, CRUD, report reconciliation, export/import and responsive UI.

! Failure diagnostic

Vibe-coded output drifts across files → freeze interfaces, test after each change and keep a decision/prompt log.

NATIVE POWERPOINT CHART

Topic 04 · Evidence Chart



What the data shows

The chart is editable and bound to the same metric grain as the SQL. Validate totals before interpreting differences; visual polish never substitutes for reconciliation.

RECAP

Topic 04 · Decision Checklist

1

Contract

Can you state grain, keys and rules?

2

Mechanism

Can you trace runtime row flow?

3

Control

Can you explain the failure path?

4

Evidence

Can you reproduce and reconcile output?

COURSE OUTCOMES

What You Built

1

11 web labs

Each folder runs independently with HTML/CSS/JS + SQLite WASM

2

Prompt log

Requirements, AI output, corrections and decision evidence

3

SQL portfolio

DDL, CRUD, joins, aggregates, CTEs, transactions and plans

4

Capstone

Service operations app with dashboard and acceptance checks

Practice Beyond Class

1

Rebuild

Delete generated query and recreate it

2

Perturb

Change data and test boundaries

3

Explain

Narrate row flow and controls

4

Review

Use the Tertiary Exams portal for SQL-adjacent data practice

READY CHECK

Final Assessment

1

WA

Explain K1–K7 using course mechanisms

2

PP

Model, query, aggregate and map one scenario

3

Submit

Upload only candidate evidence to LMS

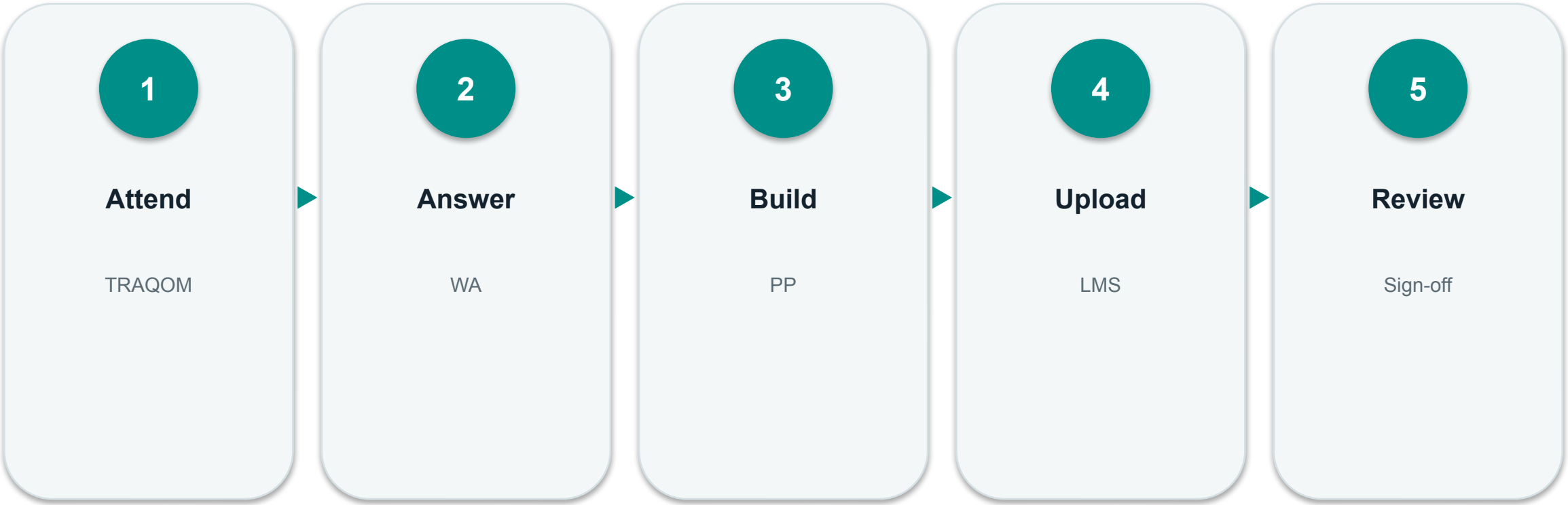
4

Privacy

Answer keys stay trainer-private

FINAL CHECK

Assessment Flow



CLOSE-OUT

Digital Attendance

1

Assessment QR

Complete before starting

2

Submission

Confirm LMS upload

3

Feedback

Complete TRAQOM survey

4

Certificate

Issued after attendance + competency

Thank You

Turn intent into tested SQL.

Model · Query · Validate · Ship